PITWALL — An Intelligent F1 Opponent Prediction and Counter-Strategy Engine

1. Project Overview and Context

We are a team of three university students participating in a hackathon running from **26 October to 4 November 2026**. We are developing a Formula 1 race strategy intelligence system called **PITWALL**, designed to predict an opponent's likely pit-stop decisions and recommend optimal counter-strategies for our own racing team.

The project combines machine learning, statistical inference, motorsport telemetry analysis, probability theory, time-series forecasting, Monte Carlo simulation, and decision optimisation.

Our central objective is to build a **1v1, uncertainty-aware, opponent-focused race strategy decision-support system**.

Rather than attempting to optimise the entire race strategy for all cars simultaneously, we will focus on the strategic interaction between two selected cars:

- **Our car:** The car whose race strategy we are responsible for optimising. We assume access to richer telemetry and tyre-condition information for this car.
- **Opponent car:** A competing car whose strategic decisions we wish to anticipate. We assume access only to publicly observable race data and limited public telemetry, not its proprietary internal telemetry or strategic plans.

The system will continuously monitor the opponent's race performance, infer their likely tyre condition, and calculate the probability that they will pit on upcoming laps.

When this probability crosses a configurable threshold, a secondary counter-strategy engine will activate. This engine will use our own car's telemetry, estimated tyre condition, and current race circumstances to determine whether we should pit immediately, remain on track, or extend our stint by a specific number of laps.

The counter-strategy engine will compare multiple possible decisions under uncertainty and recommend the action that maximises our expected strategic advantage.

The system is not intended to replace human strategy engineers. It is a probabilistic decision-support tool designed to help engineers anticipate opponent behaviour, quantify strategic risks, and evaluate possible responses.

Our proof of concept will focus on one opponent at a time, while retaining a modular architecture that could eventually be extended to multi-car strategic interactions.

---

2. Fundamental Problem Statement

Formula 1 race strategy is a sequential decision-making problem under uncertainty.

Race engineers must make strategic decisions using incomplete information about competing teams.

Although a team possesses detailed telemetry from its own car, it generally does not have access to an opponent's complete proprietary telemetry, including precise tyre temperatures, tyre wear, internal mechanical conditions, and planned pit-stop strategy.

However, publicly observable race data provides indirect evidence about the opponent's condition and intentions.

Examples include:

- Lap times and sector times.
- Current tyre compound.
- Tyre age and stint history.
- Race position.
- Gaps to surrounding cars.
- Publicly available speed and car telemetry.
- Track and weather conditions.
- Previous pit-stop behaviour.
- Recent pace deterioration or improvement.
- Safety-car and race-control conditions.

Our project investigates whether these observations can be used to infer an opponent's hidden performance state and predict their future strategic actions.

More importantly, we want to determine whether these predictions can be translated into actionable recommendations for our own team.

The central research and engineering question is:

Can we predict an F1 opponent's pit-stop behaviour from incomplete publicly observable information and use these probabilistic predictions, together with richer own-car information, to optimise our team's pit-stop response?

This question can be decomposed into three subproblems.

Problem A: Opponent State Estimation

Given externally observable race information, estimate the opponent's tyre-related performance degradation and expected future pace.

Problem B: Opponent Strategy Prediction

Given the estimated opponent state and current race circumstances, predict the probability distribution of their next pit-stop timing.

Problem C: Counter-Strategy Optimisation

Given the opponent's predicted pit-stop timing distribution and our own car's estimated performance, determine which of our possible pit-stop strategies maximises expected strategic utility.

The complete system should connect these three problems into one integrated pipeline.

---

3. Overall System Architecture

The project will consist of eight interconnected functional modules.

Module 1: Historical Race Data Acquisition and Processing

Module 2: Opponent Tyre Degradation and Pace Estimation

Module 3: Opponent Pit-Stop Probability Prediction

Module 4: Own-Car Telemetry and Tyre Performance Modelling

Module 5: Probability Threshold and Threat Detection

Module 6: 1v1 Counter-Strategy Simulation

Module 7: Strategic Decision Optimisation and Recommendation

Module 8: Interactive Race Replay, Dashboard, and Validation

The overall workflow is:

Historical F1 Data → Race State Reconstruction → Opponent State Estimation → Opponent Pit Probability Forecast → Probability Threshold Check → Own-Car Performance Forecast → Counter-Strategy Simulation → Optimal Pit Decision → Interactive Dashboard

The own-car performance model may operate continuously or update whenever new information becomes available. The threshold controls when the computationally heavier counter-strategy analysis is triggered.

All modules should communicate using a shared, structured representation of the current race state.

The system should operate on a lap-by-lap basis.

At the completion of each lap, the race state is updated, opponent predictions are recalculated, and strategic recommendations are revised when necessary.

---

4. Module 1 — Race Data Acquisition and Historical Replay

Objective

Construct a reliable data-processing system capable of reconstructing historical Formula 1 races lap by lap.

The data engine will provide the observations required by all subsequent modules.

Data Sources

The primary proposed sources are:

FastF1

A Python library that provides access to historical F1 session data, including lap timing, tyre information, weather, and selected telemetry.

Documentation: https://docs.fastf1.dev/

OpenF1

An API providing historical Formula 1 timing, telemetry, position, weather, stint, and pit-stop information.

Documentation: https://openf1.org/docs/

We may combine these sources where appropriate, subject to data availability and compatibility.

Required Data

For each selected car and lap, we aim to collect:

- Race identifier.
- Circuit and event.
- Season.
- Current lap number.
- Total race laps.
- Driver identifier.
- Car/team identifier.
- Current race position.
- Lap time.
- Sector times.
- Tyre compound.
- Tyre age.
- Current stint number.
- Pit entry and exit information.
- Gap to the selected opponent.
- Available speed, throttle, and braking telemetry.

- Track temperature.
- Air temperature.
- Rainfall or weather conditions.
- Track status and safety-car periods.

Not all fields will necessarily be available for every race.

The pipeline must handle missing data gracefully.

Historical Replay

The system will reconstruct a race chronologically.

For example, when replaying lap 32, the models must only access information available at or before the relevant decision point.

They must not access the actual pit decisions, lap times, tyre behaviour, or race events that occur after that point.

This is essential to prevent future-information leakage.

The replay system should allow a user to select a historical race, choose two competing cars, and advance through the race lap by lap.

At each step, it should reconstruct the observable state and pass it to the predictive modules.

Expected Output

A structured race-state object containing the current observable information for both cars.

This object should be consistent across all modules.

---

5. Module 2 — Opponent Tyre Degradation and Pace Estimation

Objective

Estimate how the opponent's tyre performance is evolving, despite not having access to their complete internal telemetry.

The system should estimate tyre-related pace loss rather than claim to reconstruct exact physical tyre wear or temperatures.

Fundamental Challenge

An opponent's observed lap time depends on many factors besides tyre degradation.

These include:

- Fuel consumption.
- Tyre age and compound.
- Track temperature.
- Track evolution.
- Traffic.
- Dirty air.
- Driver errors.
- Deliberate tyre management.
- Safety-car periods.
- Mechanical issues.

A slower lap does not necessarily indicate worsening tyre condition.

Therefore, the model must attempt to distinguish tyre-related degradation from other effects.

Mathematical Framework

A simplified observed lap-time model is:

$$
L_t=B_t+D_t+F_t+C_t+\epsilon_t
$$

where:

- L_t is the opponent's observed lap time at lap t.
- B_t is underlying car-driver pace.
- D_t is the estimated tyre degradation penalty.
- F_t represents fuel-related pace effects.
- C_t represents contextual effects such as traffic and track conditions.
- ϵ_t represents unexplained noise.

The objective is to estimate the degradation-related component D_t.

Because the components are not uniquely identifiable from lap times alone, the model must make assumptions and report uncertainty.

Proposed Implementation

For the initial hackathon prototype, use a relatively simple statistical model.

Potential approaches include:

- Linear regression.
- Polynomial regression.
- Ridge regression.
- Gradient-boosted regression.

- Rolling lap-time residual analysis.

Input features may include:

- Tyre compound.
- Tyre age.
- Race lap.
- Recent clean-lap times.
- Track temperature.
- Estimated fuel effect.
- Traffic indicators.

The model should predict expected clean-air pace and estimate how much tyre-related performance is being lost.

Outputs

The module should produce:

1. Estimated current tyre-related pace penalty.
2. Estimated degradation trend.
3. Expected lap times over the next several laps.
4. An uncertainty range around those forecasts.
5. An estimated useful stint extension under specified assumptions.

For example:

- Estimated degradation penalty: +0.35 seconds.
- Expected additional pace loss: +0.08 seconds per lap.
- Predicted next-lap time: 94.5 ± 0.3 seconds.

These figures are illustrative.

Important Limitation

Precise tyre surface temperature, carcass temperature, and physical wear cannot generally be reconstructed reliably from public lap timing alone.

The initial model should not claim to measure these variables.

A future version could introduce a physics-informed tyre model or additional sensor information.

---

6. Module 3 — Opponent Pit-Stop Probability Prediction

Objective

Predict the likelihood that the selected opponent will pit at the end of the current lap

or within a specified future window.

This is the central predictive component of the project.

Modelling Approach

We propose using a discrete-time pit-stop hazard model.

At every eligible lap, the model estimates the probability that the opponent will pit at the end of that lap, conditional on having reached that lap without already making the relevant stop.

Let:

$$
h_t=P(T=t\mid T\geq t,X_t)
$$

where:

- T represents the opponent's next pit-stop lap.
- t is the current decision lap.
- X_t is the information available at that decision point.
- h_t is the conditional pit-stop probability.

The model may initially use logistic regression.

$$
h_t=\frac{1}{1+e^{-(\beta_0+\beta_1x_1+\cdots+\beta_nx_n)}}
$$

The coefficients are learned from historical race data.

Input Features

Potential features include:

- Current tyre age.
- Tyre compound.
- Race lap.
- Remaining race distance.
- Number of completed pit stops.
- Estimated degradation rate.
- Recent pace deterioration.
- Current gap to our car.
- Relative pace between the two cars.
- Estimated undercut opportunity.
- Weather and track conditions.
- Race-control status.
- Historical tyre stint behaviour.

Only information available before the predicted pit decision may be used.

Training Data

Historical races will be transformed into a car-lap dataset.

Each observation corresponds to a car at an eligible pit decision point.

The target label is:

- 1 if the driver pits at the end of that lap.
- 0 otherwise.

Pit events must be aligned carefully with lap timing and data timestamps.

The model must distinguish routine strategic stops from unusual events where possible.

Multi-Lap Forecasting

The model should ideally predict not only the next-lap pit probability but also the probability of a pit stop within the next three to five laps.

For example:

$$
P(T\leq t+3\mid T>t,X_t)
$$

A multi-lap forecast cannot simply assume future conditions are already known.

A possible implementation is to project future features using the pace model and recursively estimate future pit hazards.

Under a specified future-feature scenario, the cumulative probability of a stop over the next k laps is:

$$
P(\text{Pit within }k)=1-\prod_{j=1}^{k}(1-h_{t+j})
$$

Here, each hazard is conditional on the opponent surviving without pitting through the preceding laps.

Outputs

The module should return:

- Probability of pitting on the next lap.

- Probability of pitting within three laps.
- Probability of pitting within five laps.
- Most likely pit window.
- A distribution over possible next pit laps.
- Confidence or calibration information.

For example:

Future lap	Conditional pit probability
33	12%
34	25%
35	40%
36	20%
37	10%

These values are illustrative conditional hazards, not mutually exclusive probabilities.

The module should also convert them into a proper probability distribution over the first pit-stop lap.

Validation

Evaluate predictions using:

- Brier score.
- Probability calibration.
- Log loss.
- Pit-window recall.
- Comparison with simple tyre-age-based baselines.

Training and test data should be separated by race or chronological event groups, rather than randomly splitting individual laps from the same race.

---

7. Module 4 — Own-Car Telemetry and Tyre Performance Modelling

Objective

Forecast our own car's tyre performance and future lap times using richer information than is available for the opponent.

This module represents the asymmetric-information aspect of the project.

Information Available to Our Team

In a real F1 setting, our team could potentially access:

- Detailed car telemetry.

- Tyre temperature measurements.
- Internal tyre-condition estimates.
- Brake temperatures.
- Speed and throttle traces.
- Fuel-related estimates.
- Driver feedback.
- Tyre compound and stint history.
- Team-specific performance models.

For the hackathon, real proprietary F1 telemetry is unlikely to be available.

Therefore, we will use publicly available telemetry and, if appropriate, clearly labelled synthetic private tyre-state measurements.

Synthetic measurements must never be presented as actual proprietary data.

Modelling Objective

The model should estimate our own car's expected lap times over a future horizon.

Let:

$$
\hat L_u(t+k)
$$

represent the predicted lap time of our car k laps into the future.

The prediction should depend on:

- Current tyre age.
- Current compound.
- Recent pace.
- Estimated degradation.
- Track conditions.
- Projected fuel effects.
- Whether the car pits and changes tyres.

Counterfactual Tyre Changes

The model must be able to forecast performance under alternative strategies.

For example:

Strategy A: Pit now

The car incurs pit-lane time loss but begins a new stint on fresh tyres.

Strategy B: Stay out for two laps

The car avoids immediate pit loss but continues on increasingly worn tyres before stopping.

Strategy C: Extend the stint

The car remains on track longer, potentially exploiting track position or avoiding traffic, at the cost of older tyres.

Each strategy generates a different projected lap-time trajectory.

Outputs

The module should return:

- Current own-car tyre performance estimate.
- Expected pace over the next several laps.
- Projected pace after a tyre change.
- Expected degradation during stint extension.
- Uncertainty estimates for each forecast.

The output will be consumed by the counter-strategy simulator.

---

8. Module 5 — Probability Threshold and Threat Detection

Objective

Determine when the opponent's predicted behaviour warrants activating the counter-strategy simulator.

Basic Trigger

Define a configurable threshold:

$$
\theta\in[0,1]
$$

Let:

$$
p_t=P(\text{Opponent pits within }k\text{ laps}\mid X_t)
$$

The basic activation rule is:

$$
\text{Activate simulator if }p_t\geq\theta
$$

For example, if the three-lap pit probability reaches 70% and the threshold is 65%, the counter-strategy engine activates.

The threshold should explicitly correspond to a particular forecast horizon.

A 65% next-lap probability and a 65% five-lap probability are not equivalent.

Threat-Based Override

The system should also consider the consequences of an opponent pit stop.

An opponent with a 40% pit probability might represent a more serious threat than an opponent with an 80% pit probability if the former has a much greater undercut opportunity.

Therefore, we propose an optional override based on estimated strategic loss.

A simplified trigger could be:

$$
\text{Activate if } p_t \geq \theta
\quad\text{or}\quad
\text{Expected strategic loss} \geq \lambda
$$

where λ is a configurable risk threshold.

Outputs

The trigger module should produce:

- Monitoring status.
- Activation status.
- Reason for activation.
- Estimated threat level.
- Forecast horizon.
- Relevant opponent pit scenarios.

This module is primarily rule-based and does not require a separate machine-learning model.

---

9. Module 6 — 1v1 Counter-Strategy Simulation

Objective

Evaluate how our car should respond to the opponent's predicted pit-stop decisions.

This is the main strategic decision-making component.

Scope

The initial model will consider two cars:

- Our car.
- One selected opponent.

The simulation will focus on their relative track position, lap times, tyre condition, pit timing, and race outcomes.

Other cars will not be modelled as independent strategic agents.

However, simplified traffic and rejoining constraints may be incorporated to prevent unrealistic calculations.

Candidate Actions

The system should evaluate a discrete set of possible decisions, such as:

$$
A=\{\text{Pit now},\text{Pit in 1 lap},\ldots,\text{Pit in }N\text{ laps}\}
$$

The maximum stint extension N should be configurable.

Invalid actions must be excluded based on tyre constraints, remaining race distance, and applicable race regulations.

Pit-Stop Time Loss

When a car pits, it incurs a time cost.

This may include:

- Pit-lane entry.
- Pit-lane traversal.
- Stationary tyre-change time.
- Pit-lane exit.
- Acceleration back to racing speed.

The simulator should use a circuit-specific estimate of total effective pit loss.

It must avoid double-counting pit-lane time and normal lap time.

Undercut Modelling

An undercut occurs when a car pits earlier than a competitor and gains enough time on fresher tyres to emerge ahead after the competitor eventually pits.

The simulator should estimate whether the opponent can gain sufficient time during the interval between their pit stop and ours.

The calculation should consider:

- Initial time gap.
- Opponent's expected fresh-tyre pace.
- Our expected old-tyre pace.
- Pit-stop losses.
- Tyre warm-up effects.
- Traffic and rejoining conditions.

Overcut Modelling

An overcut occurs when remaining on track longer creates a strategic advantage over a competitor who pits earlier.

This may happen if:

- The car remaining out maintains strong pace.
- The pitting car encounters traffic.
- New tyres require warm-up.
- Track conditions favour extending the stint.

The simulator should compare both undercut and overcut scenarios.

Monte Carlo Simulation

Because opponent pit timing and future lap times are uncertain, the engine should use Monte Carlo simulation.

For each candidate action:

1. Sample an opponent pit-stop scenario from the predicted probability distribution.
2. Sample future lap-time variation for both cars.
3. Apply tyre degradation and tyre-change effects.
4. Apply pit-lane time losses.
5. Simulate both cars over the relevant race horizon.
6. Calculate the resulting time gap and relative position.
7. Record the outcome.
8. Repeat across many scenarios.

A prototype could initially use approximately 1,000 simulations per candidate action, subject to runtime constraints.

Outputs

For each candidate strategy, calculate:

- Expected relative time gap.
- Probability of retaining or gaining position.
- Expected time lost or gained.
- Distribution of possible outcomes.
- Estimated downside risk.
- Remaining tyre-life implications.

These results should be passed to the decision optimiser.

---

10. Module 7 — Strategic Decision Optimisation

Objective

Select the pit-stop strategy that provides the highest expected strategic utility.

The system should not automatically recommend pitting simply because the opponent is likely to pit.

Instead, it should compare the expected outcomes of all feasible actions.

Mathematical Framework

Let:

- S_t represent the current race state.
- T_o represent the opponent's uncertain next pit lap.
- T_u represent our chosen pit lap.
- $U(T_u,T_o,S_t)$ represent the utility of our strategy.

The optimal action is:

$$
T_u^*=
\arg\max_{T_u}
\mathbb{E}[U(T_u,T_o,S_t)\mid S_t]
$$

Equivalently, for a discrete opponent pit distribution:

$$
T_u^*=
\arg\max_{T_u}
\sum_{T_o}
P(T_o\mid S_t)
\mathbb{E}[U(T_u,T_o,S_t)\mid T_o,S_t]
$$

Utility Function

For the hackathon prototype, the utility function could prioritise:

1. Probability of finishing ahead of the selected opponent.
2. Expected time advantage after the relevant pit sequence.
3. Expected time advantage at the end of the simulated race.
4. Risk of severe strategic loss.

A weighted objective may be used:

$$
U(a)=
w_1P(\text{Finish ahead}\mid a)
-w_2\mathbb{E}[\text{Time deficit}\mid a]
-w_3\operatorname{Risk}(a)
$$

The weights must be defined consistently and the component metrics normalised where necessary.

Alternatively, the system can use a single primary objective and display the remaining metrics separately.

Recommendation Format

The engine should provide a ranked comparison.

For example:

Strategy	Probability of finishing ahead	Expected relative gap
Pit now	76%	+2.1 s
Pit next lap	64%	+0.8 s
Stay out 2 laps	43%	-1.5 s
Stay out 3 laps	29%	-3.2 s

These figures are fictional examples.

The engine would recommend pitting now in this scenario, assuming the objective is maximising the probability of finishing ahead.

Explainability

Every recommendation should include a concise explanation.

For example:

"Opponent pit probability within three laps has reached 74%. Their expected fresh-tyre advantage is approximately 1.2 seconds per lap. Remaining out for two additional laps creates a high risk of losing track position. Pitting now produces the

highest estimated probability of retaining position."

The explanation should identify the assumptions and uncertainty behind the recommendation.

---

11. Module 8 — Interactive Pit Wall Dashboard

Objective

Develop an intuitive dashboard that demonstrates the complete prediction-to-decision workflow.

The interface should resemble a simplified F1 pit-wall intelligence system.

Proposed Technology

For rapid hackathon development:

- Python.
- Streamlit.
- Plotly.
- pandas.
- NumPy.
- scikit-learn.

A React frontend with a FastAPI backend is a possible alternative if the team already has the required experience.

Main Dashboard Components

A. Race Selection

Allow users to select:

- Historical Grand Prix.
- Season.
- Our car.
- Opponent car.
- Replay lap.

B. Race State Display

Show:

- Current lap.
- Current positions.
- Gap between cars.
- Tyre compounds.

- Tyre ages.
- Recent lap times.
- Track conditions.

C. Opponent Intelligence Panel

Show:

- Estimated degradation trend.
- Recent pace performance.
- Forecast lap times.
- Next-lap pit probability.
- Three-lap pit probability.
- Five-lap pit probability.
- Predicted pit window.

D. Probability Forecast Visualisation

Display a bar chart showing the probability distribution over possible future pit laps.

The chart must distinguish conditional hazards from unconditional pit-lap probabilities.

E. Threshold Control

Allow the user to adjust the pit-probability activation threshold.

Display whether the strategy engine is currently:

- Monitoring.
- Triggered.
- Simulating.
- Ready with a recommendation.

F. Own-Car Intelligence Panel

Show:

- Current tyre condition estimate.
- Estimated degradation.
- Expected future pace.
- Projected pace after a pit stop.
- Maximum modelled stint extension.

G. Counter-Strategy Comparison

Display the estimated outcomes of:

- Pitting immediately.
- Pitting next lap.

- Remaining out for two laps.
- Remaining out for three or more laps.

Use a table or chart showing expected outcomes.

H. Recommended Strategy

Highlight the action with the highest estimated utility.

Display:

- Recommended pit timing.
- Estimated probability of success.
- Expected time advantage.
- Main reasons for the recommendation.
- Key assumptions and limitations.

I. Historical Replay

Allow the user to advance lap by lap and observe how the model updates its predictions.

J. Actual Outcome Comparison

After the relevant historical event has occurred, allow the user to reveal the actual opponent pit stop and compare it with the model's earlier predictions.

Do not reveal future events before their replay time.

---

12. Historical Backtesting and Evaluation

The project must demonstrate that its predictions are meaningfully better than simple baselines.

Opponent Pit-Stop Model Evaluation

Use historical races that were not used during training.

Evaluate:

- Probability calibration.
- Brier score.
- Log loss.
- Pit-window recall.
- Performance relative to a tyre-age-only model.

Because pit stops are relatively rare events, accuracy alone is not an appropriate evaluation metric.

Tyre/Pace Model Evaluation

Evaluate:

- Mean absolute lap-time error.
- Forecast error over multiple laps.
- Error by compound and tyre age.
- Performance on clean laps versus traffic-affected laps.

Strategy Model Evaluation

Evaluate the strategy engine using controlled simulated scenarios.

Compare against:

- Fixed pit schedules.
- Simple tyre-age-based pit rules.
- Reactive strategies that pit immediately after the opponent.
- A strategy that ignores opponent pit probabilities.

Use identical simulated conditions when comparing strategies.

Counterfactual Limitations

Historical races reveal what actually happened, but they do not reveal what would have happened if our team had made a different decision.

Therefore, alternative-strategy outcomes must be presented as simulated counterfactuals, not observed historical facts.

The simulator's assumptions should be documented.

---

13. Project Scope and Constraints

We have three developers and approximately ten calendar days.

The system must therefore prioritise an integrated, functioning prototype over excessive algorithmic complexity.

Essential Features

The minimum viable product should include:

1. Historical F1 race ingestion.
2. Lap-by-lap replay.
3. Two-car race selection.
4. Basic opponent tyre-related pace estimation.

5. Opponent pit-stop probability prediction.
6. Configurable probability threshold.
7. Own-car pace forecasting.
8. Basic undercut calculations.
9. Candidate pit-stop strategy comparison.
10. Interactive dashboard.
11. Basic historical model validation.

Stretch Features

If time permits:

- Monte Carlo uncertainty propagation.
- Detailed probability distributions over opponent pit timing.
- Traffic-aware pit rejoining.
- Probability calibration visualisation.
- Synthetic private tyre-sensor data.
- Automatic strategy explanations.
- Scenario comparison and sensitivity analysis.

Features Deferred Beyond the Hackathon

Do not prioritise:

- Precise physical tyre-temperature reconstruction.
- Complete 20+ car strategic optimisation.
- Deep reinforcement learning.
- Multi-agent equilibrium modelling.
- Production-grade real-time F1 integration.
- Highly detailed tyre thermodynamics.
- Reliable tyre failure prediction.
- Full safety-car forecasting.

---

14. Proposed Division of Responsibilities

Developer 1 — Opponent Intelligence and Machine Learning

Responsibilities:

- Opponent tyre degradation estimation.
- Pit-stop probability modelling.
- Feature engineering.
- Historical training dataset preparation.
- Probability calibration.
- Model validation.

Deliverable:

A callable opponent prediction module returning pace estimates and a pit-stop probability distribution.

Developer 2 — Own-Car Modelling and Counter-Strategy

Responsibilities:

- Own-car pace and degradation forecasting.
- Pit-lane time-loss modelling.
- Undercut and overcut calculations.
- Monte Carlo race simulation.
- Candidate strategy evaluation.
- Decision optimisation.

Deliverable:

A callable counter-strategy module returning ranked pit decisions and estimated outcomes.

Developer 3 — Data Infrastructure and Dashboard

Responsibilities:

- FastF1/OpenF1 integration.
- Historical race reconstruction.
- Shared data structures.
- Interactive dashboard.
- Probability threshold control.
- Visualisations.
- System integration.

Deliverable:

An interactive application connecting both predictive and strategic components.

All developers should participate in integration testing and final validation.

---

15. Recommended Development Schedule

October 26–27: Foundations

- Establish GitHub repository.
- Agree on data schemas.
- Load historical race data.
- Build initial replay engine.
- Identify usable features and pit-stop labels.
- Create initial dashboard skeleton.

October 28–29: Core Modelling

- Build opponent degradation estimator.
- Train initial pit-stop probability model.
- Implement own-car pace forecasting.
- Develop basic pit-loss calculations.
- Implement probability threshold logic.

October 30: First End-to-End Integration

Connect:

Race data → Opponent prediction → Trigger → Strategy comparison → Dashboard.

By the end of this stage, the complete pipeline should work, even if the models remain basic.

October 31–November 1: Strategic Simulation

- Implement candidate pit timing scenarios.
- Add Monte Carlo uncertainty.
- Compare expected outcomes.
- Improve undercut calculations.
- Integrate recommendation logic.

November 2: Validation

- Test on held-out races.
- Check for future-information leakage.
- Evaluate prediction accuracy and calibration.
- Compare against baselines.
- Inspect strategy simulations for unrealistic behaviour.

November 3: Demonstration Preparation

- Select compelling historical scenarios.
- Improve visual presentation.
- Prepare architecture explanation.
- Document assumptions and limitations.
- Test the full replay.
- Prepare backup demonstrations.

November 4: Final Submission

- Verify the complete application.
- Finalise documentation.
- Prepare presentation and demonstration.
- Submit the project.

---

16. Proposed Technical Stack

Programming language: Python.

Data acquisition: FastF1 and OpenF1.

Data manipulation: pandas and NumPy.

Machine learning: scikit-learn, with optional XGBoost.

Statistical modelling: Regression and discrete-time hazard modelling.

Simulation: NumPy-based Monte Carlo methods.

Visualisation: Plotly.

Application interface: Streamlit.

Version control: Git and GitHub.

Data storage: CSV or Parquet for the prototype.

The implementation should remain modular so that models and UI components can be replaced independently.

---

17. Expected Demonstration Scenario

The final project should demonstrate the following sequence.

Step 1: Select a historical race

The user chooses a Grand Prix and two competing cars.

Step 2: Advance the replay

The application reconstructs the race up to a selected lap.

Step 3: Analyse the opponent

The system estimates the opponent's tyre-related degradation and predicts their likely future pit timing.

Step 4: Detect a strategic threat

The predicted probability of a pit stop exceeds the configured threshold, or an expected-loss override is triggered.

Step 5: Activate the counter-strategy engine

The system loads our own car's current tyre-performance estimate and generates alternative pit-stop strategies.

Step 6: Simulate possible outcomes

The engine evaluates different combinations of opponent pit timing and our own pit timing.

Step 7: Recommend a response

The application identifies the strategy with the highest expected utility.

Step 8: Reveal the historical outcome

The user advances the replay and observes when the opponent actually pitted.

The dashboard compares the historical outcome with the earlier predictions.

Step 9: Explain the model's performance

The application presents prediction accuracy, uncertainty, and any relevant limitations.

---

18. Key Engineering Principles

The following principles must guide the implementation.

1. No Future-Information Leakage

Every historical prediction must use only information available at the simulated decision time.

2. Explicit Uncertainty

Predictions should be probabilistic wherever possible.

Do not present uncertain estimates as exact facts.

3. Asymmetric Information

The opponent model should use only information legitimately available about competing cars.

The own-car model may use richer information, but synthetic measurements must be clearly labelled.

4. Modular Architecture

Each component must expose a clear input-output interface.

5. Simple Baselines First

Begin with simple statistical models before attempting advanced machine learning.

6. Strategic Relevance

Predictive accuracy is valuable only if the information can improve or inform strategic decisions.

7. Explainability

Recommendations should be understandable to users who are not machine-learning experts.

8. Realistic Scope

A reliable and validated 1v1 proof of concept is preferable to an incomplete full-grid simulation.

9. Honest Evaluation

Clearly distinguish observed historical outcomes from simulated counterfactual results.

10. Extensibility

The architecture should permit future development into a multi-opponent strategy engine without requiring a complete redesign.

---

19. Future Research and Expansion

After the hackathon, the project could evolve in several directions.

A. Multi-Opponent Strategy Modelling

Extend the system from 1v1 interactions to simultaneous modelling of multiple competitors.

B. Bayesian State Estimation

Use Kalman filters, particle filters, or Bayesian state-space models to infer latent tyre-performance states.

C. Advanced Opponent Modelling

Learn team-specific strategic tendencies, pit-window preferences, and behavioural responses.

D. Reinforcement Learning

Investigate whether reinforcement-learning agents can discover effective pit-stop policies under uncertain opponent behaviour.

E. Multi-Agent Game Theory

Model the strategic interaction between teams that continuously adapt to one another.

F. Enhanced Telemetry

Integrate richer physical tyre-state information where legitimate datasets become available.

G. Live Decision Support

Adapt the historical replay architecture to live data streams, subject to data access, reliability and latency constraints.

H. Risk-Sensitive Optimisation

Allow users to select between conservative and aggressive strategies by changing the utility function.

---

20. Final Project Vision

PITWALL is a probabilistic Formula 1 opponent-intelligence and counter-strategy engine.

Its purpose is not merely to predict when an opponent will pit.

Its purpose is to transform incomplete observations about a competitor into actionable strategic decisions.

The system follows a complete reasoning pipeline:

Observe → Estimate → Predict → Detect → Simulate → Optimise → Recommend.

It combines two distinct sources of information:

- Incomplete external information about the opponent.
- Richer internal information about our own car.

Using these inputs, it estimates an opponent's future pit-stop behaviour and evaluates how our team should respond.

The hackathon deliverable will be a working 1v1 proof of concept featuring historical race replay, probabilistic opponent pit-stop prediction, threshold-based counter-strategy activation, own-car performance modelling, pit timing simulation, and an interactive dashboard.

The ultimate research direction is a scalable, uncertainty-aware racing strategy intelligence platform capable of supporting strategic decisions in complex multi-agent competitive environments.

---

21. Instructions for an AI Assistant or Coding Agent Working on This Project

When helping us develop PITWALL:

1. Treat the preceding document as the complete project specification.
2. Assume the developers are university students with varying levels of programming and machine-learning experience.
3. Prioritise working implementations that can be completed within the hackathon timeframe.
4. Explain mathematical models and algorithms from first principles when requested.
5. Do not introduce unnecessary architectural complexity.
6. Prefer Python-based tools and libraries unless a different technology is clearly justified.
7. Maintain clear separation between opponent prediction, own-car modelling, simulation, and user interface.
8. Never use future race information in historical predictions.
9. Distinguish genuine data-driven predictions from synthetic or illustrative outputs.
10. Include uncertainty and meaningful evaluation wherever possible.
11. Explain how each new component integrates with the existing system.
12. Provide runnable, modular, understandable code when implementation is requested.
13. State assumptions explicitly and identify potential data limitations.
14. Compare complex approaches against simpler baselines.
15. Preserve the 1v1 proof-of-concept scope unless explicitly instructed to expand it.
16. Prioritise integration, correctness, and demonstration quality over unnecessary sophistication.

Primary objective: Help us deliver a functional, technically defensible, clearly explained and compelling F1 opponent-prediction and counter-strategy system by 4 November 2026.